

MCP Toolbox 数据库小白指南

12 个疑问 / 5 项能力 / 1 个真实只读边界实验

真实官方 Toolbox v1.13.1 已通过 MCP stdio 运行，26 项检查通过：两行虚构库存可读，五次针对主库的写入被拒，文件指纹不变。刻意标错 readOnlyHint 的独立可写对照仍能写入，证明提示标签不能替代执行边界。

核心边界：这里验证的是 SQLite URI mode=ro 打开的指定主数据库文件，不是 Toolbox 的 SQLite readOnly 功能，也不是全进程或所有附加数据库的只读保证。

疑问就是目录

- 01 它是数据库，还是让 AI 使用数据库的接线员？
- 02 我学过 MCP 了，为什么还要学 Toolbox？
- 03 Source、Tool、Toolset 到底各管什么？
- 04 参数绑定为什么比拼接 SQL 更稳妥？
- 05 能直接执行任意 SQL，为什么还要写专用工具？
- 06 把写工具从列表隐藏，等于禁止写入了吗？
- 07 readOnly、readOnlyHint、SQLite mode=ro 是同一件事吗？
- 08 登录成功，为什么也未必能读所有数据？
- 09 stdio 和 HTTP 只是启动方式不同吗？
- 10 看到“调用成功”，怎样知道到底发生了什么？
- 11 我只改了一行配置，结果为什么变了？
- 12 做完这次实验，可以直接连生产数据库了吗？

阅读路径：先看第 7 问，再按六步实验复跑；最后对照结论的适用范围。

这个项目给我什么

1 连接多种数据源

通过各类型 source 复用数据库连接；具体支持字段和权限须按版本、数据库检查。

2 提供预置与专用工具

开发时探索表结构；运行时用固定 SQL 与类型化参数表达业务操作。

3 按应用组织工具

把需要一起加载的工具归为 toolset，减少客户端无关能力与上下文负担。

4 接入身份与权限机制

按工具验证身份或绑定可信身份参数；配合数据库权限，不能由描述文字替代。

5 记录运行与诊断信号

支持连接管理、日志和 OpenTelemetry；真正的采集、保留与脱敏还需配置。

4 个具体用途

1 练习数据库助手

用虚构酒店表测试 MCP 列表、参数绑定和错误处理，不需要大模型或真实账号。

2 做固定业务查询

把“查订单状态”“统计当日库存”做成小而清楚的工具，并由可信身份限定数据。

3 辅助开发探索

在隔离测试库中查看表与查询数据，通用 SQL 操作保留人工监督。

4 复用工具服务

让不同 MCP 客户端或 Toolbox SDK 使用同一套经过测试的业务工具。

01 它是数据库，还是让 AI 使用数据库的接线员？

Toolbox 是工具服务器；真正存数据、执行 SQL 的仍是数据库。

想象客服助手收到“查一下上海的酒店”。模型理解需求，客户端发出工具调用，Toolbox 把已配置的查询交给数据库，再把结果传回。它不是大模型，也不会凭空知道你的表结构。项目把数据库连接、工具定义和调用接口集中起来，减少每个助手重复造接线代码。

术语落地：MCP（Model Context Protocol，模型上下文协议）约定客户端与工具服务器如何交换能力和请求；SQL 是关系数据库的查询语言。

场景：例子：输入城市只是参数，酒店表和结果来自数据库，不是模型“记住”的答案。



需求被翻译为工具调用；结果沿原路返回。

依据：S1

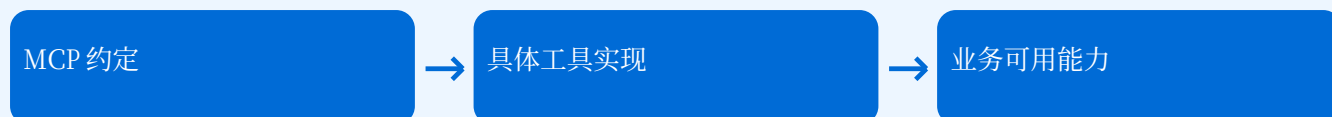
02 我学过 MCP 了，为什么还要学 Toolbox？

MCP 是接口约定，Toolbox 是实现数据库工具的一套具体软件。

把 MCP 类比成插座规格，Toolbox 是有多种数据库插口的设备。会用 MCP 客户端，不等于已经定义好数据库查询。本项目提供两条入口：开发时使用预置通用工具探索数据；应用运行时提供围绕业务设计的自定义工具。两者风险和可控范围不同。

术语落地：预置工具是项目打包好的通用能力；自定义工具是你用配置写明用途、参数和执行逻辑的能力。

场景：例子：开发者可以探索表结构；面向客户的助手通常只需“按订单号查状态”，不必获得任意 SQL。



已有 MCP 概念只复用，不把协议与产品混成一个词。

依据：S1

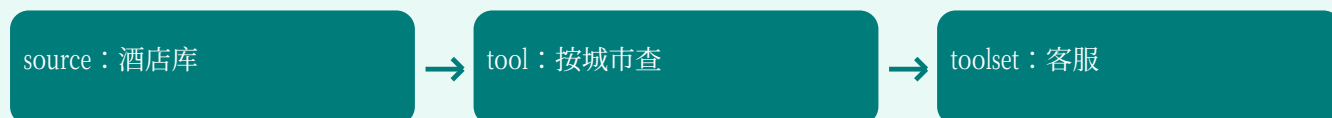
03 Source、Tool、Toolset 到底各管什么？

Source 连哪里，Tool 做什么，Toolset 一次加载哪些工具。

Source（数据源）保存数据库类型和连接信息；Tool（工具）定义可调用操作；Toolset（工具集）把工具按用途分组。比如酒店数据库是一个 source，按城市查询是一个 tool，客服常用查询放进一个 toolset。分组让客户端少看无关工具，但分组本身不能代替权限。

术语落地：配置是服务器启动时读取的结构化说明。固定文档将 toolset 视为仅含工具的 group（组）；旧配置仍兼容，但 description 等字段有例外，应核对版本。

场景：例子：同一个数据库可被多个工具复用，同一个查询工具也可出现在不同工具集中。



连接、动作、打包分层，修改时才知道该动哪一层。

依据：S1、S5

04 参数绑定为什么比拼接 SQL 更稳妥？

把输入当作值传进去，不让输入变成 SQL 结构。

sqlite-sql 的普通参数按顺序绑定到问号占位符。城市名即使含引号，也应作为城市值处理，而不是拼成新的 WHERE 条件。类型检查还能提前发现把整数传进字符串参数的错误。参数化减少值注入风险，却不能替你决定谁有权查这行数据。

术语落地：占位符是 SQL 中等待填入“值”的位置；参数绑定由驱动传递值。templateParameters 则先替换 SQL 文本，作用不同，风险也更高。

场景：例子：WHERE city = ? 可以绑定城市；不能靠 ? 动态替换表名。表名需求应优先改为固定工具或受控映射。



值进入指定槽位；不把用户文本直接当查询代码。

依据：S3

05 能直接执行任意 SQL，为什么还要写专用工具？

通用 SQL 很灵活；专用工具更容易定义、检查和限制业务范围。

sqlite-execute-sql 接收一段 SQL，适合有人监督的开发探索。sqlite-sql 则把语句固定在配置里，只开放必要参数。固定语句更便于审阅、测试和限定返回字段，但仍可能是一条写语句。工具名字叫“查询”或描述写“安全”都不构成限制。

术语落地：最小权限是只授予任务必要的能力；它必须落实在可用工具、认证授权和数据库权限等实际机制中。

场景：例子：客服只需要订单状态，不应默认能列出整张客户表。官方 SQLite 通用 SQL 文档明确提醒其不适用于生产代理。



按场景收窄输入面，再检查数据库底层权限。

依据：S4、S3

06 把写工具从列表隐藏，等于禁止写入了吗？

工具清单是可发现范围；写入权限是另一道需要实测的边界。

toolset 让某个客户端加载一组工具，减少干扰和误调用。判断安全边界时，还要问：服务器是否仍注册其他工具？其他入口能否调用？数据库连接是否允许写？本实验会比较工具列表与实际调用结果，而不是看到菜单变短就宣布数据库只读。

术语落地：发现（discovery）是询问有哪些工具；调用（invocation）是请求执行某个工具。清单里的提示不是数据库的授权判决。

场景：例子：菜单没有“修改价格”不等于厨房没有这道操作。要用受控写入测试与数据前后对照确认边界。



分别收集三层证据，不能用第一层替代第三层。

依据：S5、S6

07 readOnly、readOnlyHint、SQLite mode=ro 是同一件事吗？

不是。提示、Toolbox 功能和数据库连接方式属于不同层。

readOnlyHint 是给客户端的行为注解，不是权限证明。部分受支持数据源的 readOnly 功能会配合底层限制、工具裁剪和注解；当前文档矩阵列出 Cloud SQL PostgreSQL、Cloud SQL MySQL、AlloyDB PostgreSQL 和 BigQuery。SQLite 源没有这个 readOnly 配置字段，源码中的 IsReadOnly 返回 false。

术语落地：SQLite URI 的 mode=ro 表示该数据库文件以只读方式打开。本实验利用驱动接受的 URI，实测对该文件的写入失败；这不等于验证了上述云数据库机制。

场景：例子：实验保留写工具并真正调用它。预期写失败、数据不变；不会使用 SQL 关键词过滤器充当安全边界。

Hint：告诉客户端



readOnly：限支持源



mode=ro：本实验

SQLite 只读文件连接不自动证明工具被裁剪或注解正确。

依据：S6、S8

08 登录成功，为什么也未必能读所有数据？

身份验证说明“你是谁”；授权还要回答“你能做什么”。

Toolbox 可按工具检查令牌，也可把身份声明绑定到参数；数据库自身还可通过账号权限限制访问。应用应避免让模型随意填写他人的用户编号，再把它当可信身份。跨用户、跨租户的数据隔离必须在可信层校验，不能只写进提示词。

术语落地：认证是核实身份；授权是决定允许的动作和数据范围。OIDC 是传递身份信息的标准；claim 是令牌中经验证的身份字段。

场景：例子：用户 A 的订单工具应使用可信身份限定 A 的记录。即使请求参数写成 B，也不应扩大到 B 的订单。该身份流程本次没有配置或验证。

核实调用者身份



绑定可信用用户范围



DB 权限再限制

本实验仅用虚构数据，不访问账号、令牌或真实客户数据。

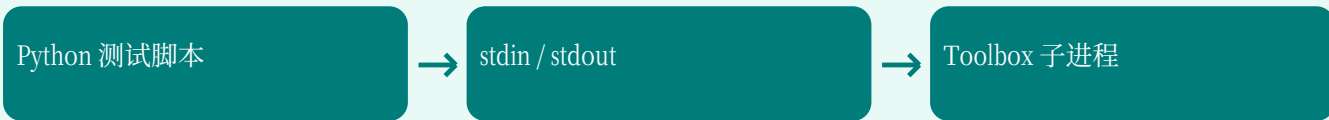
依据：S7

09 stdio 和 HTTP 只是启动方式不同吗？ 它们也是不同的连接方式和部署边界。

stdio 用进程的标准输入输出传递 MCP 消息，常见于本地客户端启动工具服务器；HTTP 通过网络端点连接，更适合独立部署但需要考虑暴露范围、认证和网络保护。本实验通过 stdio 启动官方二进制，不监听端口，不修改任何 MCP 客户端的全局配置。

术语落地：传输层是消息在客户端与服务器之间走的通道。JSON-RPC 是本实验 MCP 请求使用的消息格式，带有方法名、参数和请求编号。

场景：例子：脚本发送 tools/list，再发送 tools/call，并把实际响应记入日志。无需模型或付费 API 也能测试这个协议环节。



真实工具服务器运行；模型规划质量不在本次测试范围内。

依据：S9、S1

10 看到“调用成功”，怎样知道到底发生了什么？ 同时看响应、错误和数据库状态，才有完整证据链。

只看进程退出码会漏掉协议内的工具错误。测试应记录请求目标、参数、结果和错误，再比较数据前后状态。Toolbox 支持 OpenTelemetry 的链路指标，但部署采集器和外部后端是另一项工作。本次只保留项目内测试日志，不向遥测服务发送数据。

术语落地：可观测性是通过日志、指标和链路了解系统运行；审计记录用于回溯“谁在何时做了什么”。测试日志不等于生产审计体系。

场景：例子：本次主库读到 2 行、写调用返回只读错误、再次读取仍为 2 行，三份证据比“安全测试通过”一句话更可信。日志应避免记录令牌与真实敏感字段。



成功读取、预期拒绝、反例对照都应留下记录。

依据：S1、S10

11 我只改了一行配置，结果为什么变了？

配置也是可执行行为的一部分，需要版本、校验和回归测试。

Toolbox 可动态重载配置，因此改 source 或 statement 可能直接改变工具行为。实验固定官方二进制版本、文件 SHA-256 和运行配置，避免“昨天的文档配今天的程序”。源码 HEAD 与发布版本分开记录；遇到字段不识别，应核对该版本支持范围，不可静默删掉限制继续运行。

术语落地：版本固定是使用明确的版本或提交；SHA-256 是文件指纹，用于确认字节是否一致。它本身不证明文件来自可信发布者。

场景：例子：SQLite 源写入 readOnly: true 若被拒绝，正确结论是“该字段不受支持”，不是“只读已打开”。本实验会保留真实报错证据。



官方渠道、文件指纹、实际行为是互补证据。

依据：S1、S8

12 做完这次实验，可以直接连生产数据库了吗？

先把小实验变成清楚的验收清单，再决定部署。

本次只证明固定版本在虚构 SQLite 文件上的特定行为。生产环境还要验证目标数据库权限、租户隔离、返回数据最小化、认证、超时与资源限制、敏感日志处理和回滚方案。即使一个写操作被拒绝，也不能推断所有文件、所有数据库或所有工具都安全。

术语落地：威胁模型是列清楚要保护的资产、可能的攻击路径和信任边界；回归测试是在改动后重复检查原有保证。

场景：下一步：用脱敏测试库和真正的最小权限账号，补上越权、写入、超时、错误响应测试。只有授权和验收都明确后才考虑真实环境。



把结论带着适用范围一起迁移，避免小实验被放大。

依据：S6、S7

六步主实验

准备：Linux x86_64、Python 3 标准库。解压 ZIP 后进入 exercise 目录；先按 README 获取约 315 MB 的官方二进制，放入 vendor/toolbox。下列六条命令均已原样执行。第 2 步一次跑完真实协议测试，第 3 至 6 步逐项解读并断言实际证据。重复运行时须换一个新的输出目录。

01 核对官方程序和指纹

目的：确认实际执行的是本课固定的官方发布程序，避免把新版配置套进旧程序。

```
python3 verify_binary.py
```

实现了什么：读取 vendor/toolbox，核对 SHA-256，并实际调用 --version。程序不在包内，须先按 README 下载。

验证标准与实际结果：输出 PASS 与版本 1.13.1+binary.linux.amd64.e14cda6；完整指纹见日志。指纹是官方渠道下载后测得，不是独立发布签名。

02 一口气运行真实协议实验

目的：创建两份虚构库存库，用官方 Toolbox 子进程接收 MCP 请求。

```
PYTHONDONTWRITEBYTECODE=1 python3 run.py --output evidence/learner-run
```

实现了什么：脚本执行预置工具、只读写拒绝、参数绑定、可写负对照及不支持字段测试，并生成原始响应和配置。这里只是测试驱动，执行工具的是官方二进制。

验证标准与实际结果：终端最终 26/26 passed；results.json 的 status 为 passed。输出目录必须此前不存在；不允许覆盖旧证据。

只用项目目录内的虚构库存，stdio 不监听端口；没有账号、凭据、生产数据库、付费模型、外部遥测或全局客户端配置。错误提示标签仅为负对照实验，不能抄到生产配置。

继续复跑并核对证据

03 看见工具与两行真实结果

目的：将“工具已发现”和“查询已执行”分别核验。

```
python3 inspect_results.py evidence/learner-run discovery
```

实现了什么：读出真实清单、表发现和 SELECT 响应。工具为 execute_sql/list_tables；库存 apples=4、pears=7。

验证标准与实际结果：4 项选中检查通过；读取 2 行。注意 mode=ro 并未自动把 execute_sql 的 readOnlyHint 改成 true。

04 观察五次写拒绝和数据不变

目的：验证底层文件连接拒绝写入，而非依赖 SQL 关键词过滤。

```
python3 inspect_results.py evidence/learner-run boundary
```

实现了什么：展示五个写尝试的 MCP 错误，以及写后查询和主库文件指纹比较。PRAGMA 重置组合也未能写入。

验证标准与实际结果：7 项选中检查通过；isError=true，错误文本含 readonly database (8)；仍为 2 行，总量 11，文件 SHA-256 不变。此结论仅针对主库。

继续复跑并核对证据

05 改变输入形状并核对错误

目的：分清参数绑定、类型校验和未注册工具三种不同边界。

```
python3 inspect_results.py evidence/learner-run binding
```

实现了什么：展示已运行的 apples、SQL 外观字符串、整数 123 与未配置 execute_sql 请求的不同结果。

验证标准与实际结果：5 项选中检查通过：普通名称返回 1 行；SQL 外观值返回空；整数返回工具错误；未知工具返回 JSON-RPC -32602。不把未知工具错误说成用户鉴权通过。

06 用可写负对照拆穿提示标签

目的：证明假只读注解不阻断写入，并确认不支持的配置会失败。

```
python3 inspect_results.py evidence/learner-run control
```

实现了什么：展示故意标 readOnlyHint=true 的可写独立库新增第 3 行；SQLite readOnly:true 报 unknown field；最后再次核对主库指纹。

验证标准与实际结果：5 项选中检查通过；完整运行仍为 26/26。归档 results.json、commands.json、mcp-transcript.jsonl、配置与 stderr；没有把预期拒绝当作意外失败。

真正得到哪些证据

- 预置 sqlite/sqlite_database_tools 实际列出 execute_sql 与 list_tables；主库返回 apples=4、pears=7，共 2 行，总量 11。execute_sql 的 readOnlyHint 仍为 false。
- INSERT、UPDATE、DELETE、CREATE TABLE，以及 PRAGMA query_only=OFF 后尝试 INSERT 的组合均返回工具错误；原始文本含 attempt to write a readonly database (8)。随后读取成功，主库 SHA-256 前后相同。
- 固定 SQL 的 name=? 查询返回 apples；SQL 外观输入仅作为值，返回空结果；整数参数触发类型错误。调用未配置工具返回 -32602 和 tool does not exist，不是身份鉴权证明。
- 可写独立对照故意标 readOnlyHint=true，INSERT 仍成功，另一数据库连接确认记录从 2 行变为 3 行。SQLite source 的 readOnly:true 则启动失败，报 unknown field。
- 本次没有测试云数据库 readOnly、OAuth、HTTP 入口、租户隔离或所有 SQL 副作用。BigQuery protected 模式在文档中允许会话临时写入，不能概括为绝对零副作用。

已有知识如何复用

- 查重：公开索引 394 条元数据；全文读取并核对 5 篇，其余 389 篇没有算作已读。另完整读取网站既有 Ponytail 与 HowToLiveBetter 两篇，共 9 块。
- 复用《MCP协议》《只读闸门三原则》《本地MCP端点安全四道门》《代码管边界提示词管判断》《关系型数据库与NoSQL》；不把已有的 MCP 和最小权限概念重新包装为新知。
- 新增提案优先合并到旧概念：工具注解与强制执行分离、适配器级只读能力矩阵、工具暴露范围与数据权限分层。已有笔记不代表本人已经掌握，也不代表查重覆盖最新本机 vault。

待本机收尾

知识笔记仅为入库提案；没有修改 Windows vault，没有 Git 提交或推送。待本机回迁后，与最新笔记做同义查重和关联。任务是否完成由本人确认。

实际运行：2026-10-05T04:49:26Z 至 2026-10-05T04:49:27Z

Linux x86_64 / Python 3.12.14 / Python sqlite3 3.53.1（仅建库/复核，不代表 Toolbox 内嵌 SQLite 版本） / Toolbox 1.13.1+binary.linux.amd64.e14cda6

来源与复核入口

源码观察固定到 a24e5e68567fa014a42fc6cc711faa82b964d4c0；实际二进制为 v1.13.1，对应发布提交 e14cda6b4f483e6b5e5ec23e342f9f232a6ce60f。二者分别记录，不能把主分支文档当作发布版已测行为。

S1 项目与安装

README.md

S2 SQLite 数据源

docs/en/integrations/sqlite/source.md

S3 参数化 SQLite SQL

docs/en/integrations/sqlite/tools/sqlite-sql.md

S4 通用 SQLite SQL

docs/en/integrations/sqlite/tools/sqlite-execute-sql.md

S5 工具集

docs/en/documentation/configuration/toolsets/_index.md

S6 只读支持矩阵

docs/en/documentation/configuration/security/read-only.md

S7 身份验证

docs/en/documentation/configuration/authentication/_index.md

S8 SQLite 源实现

internal/sources/sqlite/sqlite.go

S9 MCP 客户端连接

docs/en/documentation/connect-to/mcp-client/_index.md

S10 遥测导出

docs/en/documentation/monitoring/export_telemetry.md

上方来源标题可点击，均指向固定提交。完整 URL、许可证与本地证据清单同时保存在 HTML 和练习 ZIP。

产物检查

PDF 由 ReportLab 直接生成，按 A4 排版并逐页渲染检查。HTML 有 12 个疑问和 12 张内联 SVG，仅通过结构检查，没有声明浏览器视觉验收。练习 ZIP 不含大型二进制，提供官方来源、实测指纹、配置、测试脚本与复跑结果。

上游许可 Apache-2.0。引用与随包源码保留原始许可证及版权声明；原创讲解、虚构数据与测试脚本不代表上游官方保证。